EDISOFT
DEFENCE & AEROSPACE TECHNOLOGIES

embedded brains

CISTER
Research Centre in
Real-Time & Embedded
Computing Systems

jenaoptronik

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Lero THE IRISH SOFTWARE RESEARCH CENTRE

RTEMS SMP
Ready to Fly

FV-200 Formal Verification Plan (Algorithms, Requirements, Approach)
Release 4

CONTENTS

1 Introduction 3

2 OVERVIEW 5

3 METHODS 7
3.1 Formal Methods and Testing . . . 7
3.2 Formalisms and Tools . . . 9
3.2.1 Promela/SPIN . . . 9
3.2.1.1 Automation . . . 9
3.2.1.2 Test Generation . . . 9
3.2.1.3 Code Handling . . . 10
3.2.2 TLA+/PlusCal . . . 10
3.2.2.1 Automation . . . 10
3.2.2.2 Test Generation . . . 11
3.2.2.3 Code Handling . . . 11
3.2.3 Frama-C . . . 11
3.2.3.1 Automation . . . 11
3.2.3.2 Test Generation . . . 12
3.2.3.3 Code Handling . . . 12
3.2.4 Isabelle/HOL . . . 13
3.2.4.1 Automation . . . 13
3.2.4.2 Test Generation . . . 13
3.2.4.3 Code Handling . . . 13
3.3 Probabilistic Methods . . . 13
3.4 Handling Assembler . . . 14
3.5 Conclusions . . . 14

4 ALGORITHMS 15
4.1 RTEMS-SMP Code . . . 15
4.1.1 Scheduling Algorithms . . . 15
4.1.2 Context Switching . . . 16
4.1.3 Synchronisation primitives . . . 16
4.1.4 Key data-structures . . . 17
4.1.5 RTEMS API Specifications . . . 18

5 INTEGRATION 19
5.1 Integrating with Toolchain/DataPack . . . 19

5.2 RTEMS Community Integration . . . 20
5.2.1 Code Annotation . . . 21
5.2.1.1 Frama-C Annotations . . . 21
5.2.1.2 Annotation Schemes . . . 21
5.2.1.3 Community Feedback . . . 22
5.2.2 Docker . . . 22
5.2.2.1 Using Docker . . . 22
5.2.3 Future CI Issues . . . 23

6 PLANNING **25**
6.1 Task 3.2 Overview . . . 25
6.2 DDR Deliverables . . . 26
6.3 CDR Deliverables . . . 26
6.4 The Plan . . . 27
6.5 Timeline . . . 27

7 APPENDIX **29**
7.1 Conference Call with CEA . . . 29
7.1.1 Attendance . . . 29
7.1.2 Summary . . . 29
7.1.3 Follow-Up . . . 30
7.2 Context-Switch Assembler . . . 30
7.3 Sources Using Atomics . . . 31
7.4 Code Annotations . . . 32

Bibliography **35**

Identification, Copyrights and License

© 2021 Trinity College Dublin

The copyright holders listed above grant that this document may be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, under the Creative Commons Attribution-ShareAlike 4.0 International Public License.

Release	Git hash	Date	Status	Changes
01	2294e5fn	Oct 15, 2019	Replaced	Task 3.1 deliverable
02	e042c0c	Nov 08, 2019	Replaced	updates for RIDS PDR-MV-11, PDR-MV-12, PDR-TT-02, Applied RID PDR-TT-03, action 205 with the correction of spelling errors
03	d05b9459	Oct 14, 2020	Replaced	Adjust for removal of Doorstop (Issue #447)
04	35a335dc	Jan 6, 2021	Approved	Updates license as agreed by consortium

	Name	Company	Signature
Written by	Andrew Butterfield	Trinity College Dublin	
Verified by	Andrew Butterfield	Trinity College Dublin	
Approved by	Nuno Ramos	EDISOFT	

	Name	Signature (if necessary)
Approved by customer (if necessary)		

CHAPTER ONE

INTRODUCTION

This document defines a roadmap for how formal methods will be used to support the development of a top-quality qualification framework for RTEMS-SMP. Key decisions have to be made regarding the subset of the code that will be considered, the formalisms used, and the precise methodologies to be applied.

The rest of this document is structured as follows: Section 2 gives an overview of the intended outcomes for Task 3, as drawn from the SoW, negotiation and KO minutes; in Section 3 we describe the range of formal methods and tools that will be suitable for achieving Task 3 objectives; in Section 4 we discuss possible candidate algorithms to be studied; Section 5 considers how best to ensure that Task 3 tools and their outputs can be integrated with the results of Tasks 1 and 2; and in Section 6 we discuss plans for Task 3.2 in particular.

CHAPTER TWO

OVERVIEW

The output of Task 3.1 (this document) was described in the SoW as:

In the proposal it was described as:

Tasks

- To determine the scope of the formal verification effort.
- To identify the tools and approaches to be used for formal verification
- To develop a proposal with the RTEMS community about how best to handle code annotations.

The main purpose of Task 3 can be summarized as the use of formal methods to assist in ensuring that RTEMS-SMP pre-qualification tests of critical parts are focused on what are key correctness properties. A series of concrete decisions need to be made in this regard:

1. We need to decide precisely which parts of the RTEMS-SMP code are to be considered
2. We need to decide what formalisms and tools we will employ
3. We need to consider how to deploy tools in a way that fits with the RTEMS community.

The overall aim of the activity is to have a toolkit that generates all verification and validation evidence as documents that satisfy the ECSS-E-ST-40C [ECS09] and ECSS-Q-ST-80C [ECS17] standards, along with datapacks of such evidence for the target configurations running the example application. Also required is documentation to allow others to reproduce this for their own applications. For this to be workable, it is very important that as much as possible of the datapack generation is automated.

One role for task 3 is to assist in deploying static analysis tool within the RTEMS development process, so that it will become a permanent part of their methodology even after this activity has completed. Again, automation and integration into the build process will be key.

A key focus will be on the relationship between formal methods and testing [Gau95] [Gau05] [HBB+09]. The goal is to explore obtaining complete test suites from formal specifications, automating test result interpretation, and generation of observer processes.

CHAPTER THREE

METHODS

We present here an overview of a range of formal methods and tools that look feasible for use in Task 3.2.

A key criterion for any proposed tool is the ability to deploy it in a highly automated manner. This amounts to the tool having a command-line interface that covers all the features we require. One such feature is that the tool generates output that can be easily transformed into the formats used as part of the qualification datapack reports. Tools with GUI interfaces can be very helpful while developing and deploying formal models, as long as the models/tests/proofs can be re-run automatically via the command-line.

Other important criteria concern the support available for for test generation support, and how close we can connect the formalism to actual C code.

Apart from the above, there is one criteria that is a definite show-stopper: any tool we add to the tool-chain needs to be open-source, in order to be acceptable to the RTEMS community.

We start with a general overview of formal methods and testing, and discuss a number of formalisms and tools against the criteria above.

3.1 Formal Methods and Testing

A good survey of formal techniques and testing is found in a 2009 ACM survey paper [HBB+09]. They idea is that a formal specification of some form may be able to support testing in a variety of ways. They divide formal specification languages into the following groups:

Model-based: e.g., Z, VDM, B

Finite State-based: e.g., finite-state machines (FSMs), SDL, Statecharts, X-machines

Process Algebras: e.g., CSP, CCS, LOTOS

Hybrid:

mixing continuous and discrete descriptions, e.g., CHARON, here considered out of scope.

Algebraic: e.g., OBJ, CASL

In terms of how to use formal methods and tests, they suggest that automation can be provided by either: using formal specifications to generate test cases; or deriving provably correct ora-

cles for test result checking. They also point out that executable formal models can be tested themselves by model-checking.

In terms of the formal foundations of the relationship between specification and tests, they give this some discussion, citing Marie-Claude Gaudel [Gau95] as a key article. In it she identified how certain key assumptions, called *test selection hypotheses*, are important.

For each language group above, they describe test-generation techniques associated with that group.

Model-based:

> Input-domain partitioning methods, usually relying on a uniformity hypothesis; Automation often uses disjunctive normal forms of predicates. Also used are domain propagation techniques that look at boundaries on the behaviour of basic operators and functions. Various other techniques including formal refinement, mutation, have also been proposed. They conclude that these formalisms are good for test oracles, but can be hard to use for test generation, often requiring theorem proving support.

Finite State-based:

> Based on definitions of *conformance* defined in terms of the language of the corresponding formal automaton. Usually involves a test hypothesis or fault-model, that bounds the number of states that need to be used. There are a number of techniques for test generation, and they start with complete deterministic FSMs, then consider those that are partial, and finally those that are non-deterministic. Deterministic FSMs are conceptually straightforward, while partiality has a number of differing semantic interpretations when transitions are missing. A common issue is often how to minimize the resulting test suite. An interesting sub-class is when a system is modelled as a number of smaller FSMs connected together, rather than one big one. The main message here is that test generation is relatively easy, with the main problem being how to avoid combinatorial explosions.

Process Algebras:

> They have labelled transistion-systems (LTS) as semantics, although these can often be infinite, so some form of space reduction is needed. In many respects the testing has similar advantages and disadvantages as is found for FSMs.

Algebraic:

> These are described as being very suitable for OO testing. Test are either generated on the basis of operation syntax, or from the axioms. Tests based on syntax simply produce larger and larger terms built from that syntax and check that they execute correctly. Tests based on axioms replace axiom variables by larger and larger terms, and then test that evaluating the many axiom instances so produced always results in a true result.

A discussion of the automated reasoning tools indicates that these can indeed support testing. Model checkers produce counterexample witnesses, and these can be converted into tests. Tem-

poral logics describe properties that can again be converted into test sequences. In [HBB+09] they clearly state:

> "The most important role for formal verification in testing is in the automated generation of test cases. In this context, model checking is the formal verification technology of choice; this is due to the ability of model checkers to produce counterexamples in case a temporal property does not hold for a system model."

3.2 Formalisms and Tools

3.2.1 Promela/SPIN

A technique mentioned based on the SPIN model checker, with its modelling language called Promela (spinroot.com). Promela is quite a low-level modelling language that makes it easy to get close to code level, and is specifically targeted to modelling software. It is one of the most widely used model-checkers, both in industry and education. It also has a tool called modex that will automatically generate a Promela model from C code. It uses linear-time temporal logic (LTL) to express properties of interest.

It is open-source, and very easy to install, needing only cc and lex/yacc.

3.2.1.1 Automation

The `spin` program is command-line driven, so it can be easily automated. It's output is plain text reporting with some structure. We would need to decide how these get transformed into a form suitable for the datapack.

It has a GUI interface as well which may assist in developing Promela models. These can always be subsequently exercised from the command-line.

3.2.1.2 Test Generation

Given a Promela model that checks successfully, we can generated tests for a property P by asking Spin to check its negation. There are ways to get Spin to generate multiple counterexamples, as well as getting it to find the shortest. A tool called TorX was developed by the University of Twente to produce tests [dVT00], released under the Apache License, but no longer seems to be downloadable. Another system, *ScenTest* uses Promela to model and analyse test *scenarios* which are then translated into tests for a Java implementation [UAHB10]. The tool requires Sparx Enterprise Architect, which is commercial. Promela/SPIN along with LTL has been used to model/test a multi-core RTOS called AUTOSAR [FKDO12]. Key there is keeping the Promela models close to the actual run-time environment.

3.2.1.3 Code Handling

There is a program called `modex` that extracts Promela models from C code. It assumes that the programmer is using a well-known thread library such as pthreads, so may not suit our needs. Modex will take `assert()` statements in C code and bring those into the Promela model, making it possible to have code annotations that are available for interpretation by Spin.

However in a test using `rtems/cpukit/score/src/chain.c` we observe the following:

```
> verify chain.c

    Extract Model:
    ==============
modex chain.c
MODEX Version 2.11 - 3 November 2017
chain.c:27: Error (syntax error, unexpected STAR, expecting RPAREN or COMMA) before '*'
  Chain_Control *the_chain,
                ^
1 errors
modex: cannot happen fct decl1
```

Some of the tools like modex above, and Frama-C later, report syntax errors on RTEMS code. This may be due to differences in the syntax allowed between C99 and C11. Whether or not RTEMS code should be, or can be, modified to use a conservative coding style that satisfies both C99, C11, where possible, is something that should be discussed. This is related to the view in the SoW that we should adopt the NASA/JPL approach to code that fails static analysis, even if shown to be a false positive. This is because these indicate some form of "code smell" and the code should be re-written so that the tool no longer reports an error.

3.2.2 TLA+/PlusCal

The formalism Temporal Logic of Actions (TLA), is comprised of the specification language TLA+, an algorithmic language called PlusCal, and various tools (tlaplus.net). The specification language uses simple mathematics to specify concurrent systems. The PlusCal language looks like pseudo-code, but has a direct translation into TLA+, so lowering the barrier to be able to produce specifications. Tools include both a model-checker and a theorem prover.

It is open-source, easy to install, but does require the Java Runtime Environment (JRE1.8+)

3.2.2.1 Automation

The TLA+ model checked (`tlc`) has a command-line interface. It is not clear if such a thing is available for the prover. The documentation for it says it should be used from the TLA+ Toolbox IDE.

3.2.2.2 Test Generation

Again, should be possible using `tlc` as it is a model checker. However, the output when a failure is found is a sequence of predicates describing states, commented with code line number and name of function. It is not clear how this could be turned into a test.

3.2.2.3 Code Handling

There is no automatic extraction of TLA+ from C. There is a modelling language called PlusCal that helps building models close to the level of a C program, but this must be built by hand. Another tool translates PlusCal into TLA+.

3.2.3 Frama-C

Frama-C (frama-c.com) is a platform supporting a range of tools for analysing C code, including static analysers, support for functional specifications (ANSI-C Specification Language – ACSL), and links to theorem provers. Some of its analyses require code annotations, while other can extract useful information from un-annotated code. It has a plug-in architecture, which makes it easy to extend.

Frama-C, and its plugins, are implemented in OCaml, and it is installed using the `opam` package manager.

A tele-conference call was held on 18th September with two members of the Frama-C team, Florent Kirchner and Loic Sorrenson. Notes of what was discussed are in Appendix 7.1. A follow-up email interchange asked about the use of the WP plug-in by Airbus as part of their CI flow. We shall refer to some of what was said below, citing the "Frama-C team".

3.2.3.1 Automation

Frama-C has a very comprehensive command-line interface, and it has clearly been designed to fit into an automated test/verification environment. It also has a GUI which is not only a wrapper around the command-line program, but also takes any proof you build through the GUI and outputs it in a form that allows it to be replayed via the command-line.

In [BDE+18] there is a discussion of how Frama-C is used as part of the Airbus development process. Airbus have spent a decade now at a project to formalise most if not all of their software development process, by describing all engineering artifacts using notations with formal semantics, and building interoperable tools that use these notations. In particular, they can perform verification of any system with a mix of testing and proof, an approach they call "hybrid verification". In particular they have a design language DCSL that can automatically produce ACSL annotations for C programs, which themselves are produced from templates generated by their toolset. The WP plugin is used to prove the correctness of the code, typically handling 95% of the proofs automatically. Proof engineers supply loop invariants and use tactics to prove the remaining 5% of proofs. These tactics become part of the automated proof checking. As far as test design is concerned, a lot of automation handles the test boilerplate, leaving the test developer to focus on test scenarios.

An extended version of this paper can be found at http://www.di.ens.fr/~delmas/erts18/.

There is an issue about integrating tools like Frama-C into the current systems used by `rtems.org` for CI, see Section 5.2.3.

3.2.3.2 Test Generation

There is a Frama-C plugin called `pathcrawler` that does test generation, but is only available by email request for research/evaluation purposes. An online demo version of it is available at

http://pathcrawler-online.com:8080

When asked, the Frama-C team replied that they don't make it available as, in their opinion at least, using higher-level design artefacts (e.g. an ACSL specification) to automatically derive tests for a lower level design artifact (e.g., C code) is forbidden by DO178C. Hence, they have not released it formally, nor has much work gone into maintaining it. There is an analysis of other standards, including DO178C, in deliverable QT-109 [eb19a] but the issue of automatic *test* generation does not seem to be discussed.

A recent study looked at how to add model-checking to Frama-C based on counterexample guided-refinement (CEGAR) [SP16]. It developed a Frama-C plugin called *cegarmc*, which takes C annotated with ACSL and converts it to a form that can be passed to CEGAR tools (e.g. SATAB, Blast). All are freely available. However it is not clear how to get to tests from there.

3.2.3.3 Code Handling

Frama-C is designed to handle C99 from the very beginning. However, the RTEMS sources also use C11, and so there may be issues with certain parts of the code.

For example, having run the following:

```
> sparc-rtems5-gcc -E -C -Iinclude -Iscore/cpu/sparc/include score/src/threadqenqueue.c
```

to obtain *threadqenqueue.i* to pass to Frama-C, we get a syntax error:

```
> frama-c threadqenqueue.i

[kernel] Parsing threadqenqueue.i (no preprocessing)
[kernel] /users/staff/butrfeld/rtemsSMP/rtems/5/lib/gcc/sparc-rtems5/7.4.1/include/
↪stdatomic.h:40:
  syntax error:
  Location: line 40, between columns 8 and 16, before or at token: _Bool
  38
  39
  40    typedef _Atomic _Bool atomic_bool;
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  41    typedef _Atomic char atomic_char;
  42    typedef _Atomic signed char atomic_schar;
[kernel] Frama-C aborted: invalid user input.
```

Again, this is an issue where we may need consider if we can make this code more C99 compliant. This was discussed with the Frama-C team, and they said that the syntax issues are easy to fix, but the real challenge is how to use ACSL to model atomic behaviour and the underlying semantics that will drive the WP logic.

Stop Press

As of September 13th 2019, Frama-C has a release Frama-Clang 0.0.7 which supports C++. It's a prototype right now.

3.2.4 Isabelle/HOL

Isabelle/HOL is a wide-spectrum theorem-prover, implemented as an embedding of Higher-Order Logic (HOL) into the Isabelle generic proof assistant (isabelle.in.tum.de). It has a high degree of automation, including an ability to link to third-party verification tools, and a very large library of verified mathematical theorems, covering number and set theory, algebra, analysis. It is based on the idea of a small trusted code kernel that defines an encapsulated datatype representing a theorem, which can only be constructed using methods in the kernel for that datatype, but which also scales effectively regardless of how many theorems are proven. It is implemented using *polyml*, with the IDE implemented using Scala, is open-source, and is easy to install.

3.2.4.1 Automation

Proof development in Isabelle/HOL is best done using their IDE. However, all proofs can be replayed from the command-line.

There is an issue about integrating tools like Isabell/HOL into the current systems used by `rtems.org` for CI, see Section 5.2.3.

3.2.4.2 Test Generation

Currently not aware of work in this space.

3.2.4.3 Code Handling

There are encodings of the semantics of C in Isabelle/HOL, most notably that developed by NICTA/Data61 for their verification of the sel4 kernel [KEH+09]. However, that semantics is highly tailored towards their kernel code, and both were developed in tandem to ensure interoperability.

3.3 Probabilistic Methods

There is quite a lot of activity in the field of *probabilistic model-checking* (PMC) [KNP02] [HeraultLMP04] [FGT11] with a good survey in [AP18] of *statistical model-checking* (SMC).

PMC is exhaustive, with probabilities on edges, while SMC does sampling (simulations) and uses statistical inference to estimate the answer that PMC would give.

A good overview is available at https://project.inria.fr/plasma-lab/statistical-model-checking/

At this point we have not done much more investigation of these tools. At the very least, they need at least as much modelling as is required to use more "functional" model checking, such

as provided by Promela/SPIN. This is because probabilistic models are basically functional ones where transitions are annotated with probabilities. We can envisage experiment and exploration with these tools being built on top of more functional prior modelling done with classic model checkers.

3.4 Handling Assembler

One issue that has been raised is that of using formal methods to assist in determining test coverage for assembly language. The idea is we can take an assembler test-suite, and produce a tool, driven by a formal instruction-set architecture (ISA) model that computes code coverage. Magnus Myreen [MGS12] and Anthony Fox [Fox15] have developed ISA models that cover a wide range of processor designs, such as x86 and ARM. They describe a process called "decompilation" that converts machine-code into Hoare triples over the hardware state, expressed in HOL4. Fox also introduced the L3 domain specific language to specify ISAs. However the one architecture they don't treat is that of SPARC processors. A treatment of the SPARCv8 architecture has been done in Isabelle/HOL by Zhe Hou and colleagues [HSananT+16] which uses the LEON3 architecture as a concrete example. This provides an executable formalisation of the integer unit along with the register windowing, and various aspects of memory access, along with simple caching and trap models. Not covered are hardware signals and interrupts, or the modelling of concurrent behaviours at the ISA level. They can export to OCaml for execution, but the non-determinism induced by concurrency makes it difficult to write HOL models (functions) that satisfy the limitations on code generation. The resulting work has also been submitted to the Archive of Formal Proofs (AFP) [HSananTL16].

All of the above artefacts are freely available.

If we were to look at assembler coverage in Task 3.2, then the Isabelle/HOL SPARCv8 model would be a starting point. This would not be cheap in terms of effort, and raises the question: would it not be easier to get coverage data from a SPARCv8/LEON4 simulator?

3.5 Conclusions

We have surveyed a number of formal techniques to assess their ability to support testing, and to capture the behaviour of the actual C code. What is clear is that there is no one formalism/tool that covers everything. We may need to carefully tailor the tools, and abstraction level used, for any given algorithm that we choose for formal verification or testing.

CHAPTER

FOUR

ALGORITHMS

4.1 RTEMS-SMP Code

At present there are the following suggestions for the "code coverage" of the formal verification:

1. Scheduling algorithms
2. Context switching
3. Synchronisation primitives
4. RTEMS API Specifications
5. Key data-structures

Some of these came from the initial proposal, while others emerged after consultation with the RTEMS community. That email thread is accessible at

https://lists.rtems.org/pipermail/devel/2019-August/027536.html

We discuss each of these suggestions in more detail below.

4.1.1 Scheduling Algorithms

The scheduler in RTEMS-SMP includes aspects of both MrsP [BW13] and OMIP [Bra13]. Neither paper gives a clear specification of the relevant algorithm, consisting instead of a general description, with requirements and theorems mainly about the new/improved features of the algorithms, most notably aspects such as their schedulability. An early task will be to try and tease out requirements and/or specifications for what is actually implemented in RTEMS-SMP. In addition, we need to consider the appropriate level of abstract for modelling the behaviour of the target platform. This will provide a series of (formalised) assumptions that will be the basis for reasoning about code behaviour. In particular, we need to know about aspects specific to the move to multicore: cache behaviour, weak memory models. What guarantees does the LEON platform give?

At present, some relevant source-code files have been identified:

https://git.rtems.org/rtems/tree/cpukit/score/src/threadqops.c

https://git.rtems.org/rtems/tree/cpukit/score/src/threadqenqueue.c

https://git.rtems.org/rtems/tree/cpukit/score/src/scheduleredfsmp.c

4.1.2 Context Switching

The key issue with context switches is that key parts are implemented with assembler rather than C. It is difficult to automate the code coverage analysis for tests of this code. The SPARC assembler source files that are involved are listed in Appendix 7.2.

> "I am not sure what you want to do with them. You need a model of the SPARC processor, instruction set architecture and the application binary interface (ABI) to do anything with it. The SPARC architecture is quite complicated with the windowed registers." (SH)

> "The reason is simple, the GCOV, if we will use the GCOV, does not provide assembly coverage only C code. Or we get an automatic way to verify the coverage (statement and decision) (formal methods/debugging/gcov/whatever), or we will have to make it manually every time we make a release. Believe me ... making manual verification of the coverage is quite annoying." (HS)

There has been work on formalising processors at or around the ISA level, most notably by Magnus Myreen [MGS12] and Anthony Fox [Fox15]. Their work involves a neutral ISA/HW description language called L3, and covers ARM, x86 and PowerPC architectures, with some recent work on RISC-V. Their work is mainly done using HOL4 and Hoare triples specialised for assembler. There is no indication that a L3 model might exist for SPARC architectures.

David Sanan, who worked for me on the MTOBSE project, has, with colleagues in Singapore, developed a formal model of SPARCv8 in Isabelle/HOL [HSananTL16], [HSananT+16], which is now in the Archive of Formal Proofs (https://www.isa-afp.org/entries/SPARCv8.html). They have also formalised the TSO memory model for SPARCv8, which is available from http://securify.sce.ntu.edu.sg .

4.1.3 Synchronisation primitives

Includes modelling C11 Atomics and may require probabilistic methods.

The Atomic API calls used in RTEMS are:

> _Atomic_Compare_exchange_uint/uintptr/ulong _Atomic_Exchange_uintptr/ulong
>
> _Atomic_Fence _Atomic_Fetch_add_uint _Atomic_Fetch_and_uint
>
> _Atomic_Fetch_or_uint/ulong _Atomic_Fetch_sub_uint _Atomic_Flag_clear
>
> _Atomic_Flag_test_and_set _Atomic_Init_uint/uintptr/ulong
>
> _Atomic_Load_uint/uintptr/ulong _Atomic_Store_uint/uintptr/ulong

The source files that use Atomics are listed in Appendix 7.3.

An example of easy use of atomics can be found in the following source file:

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smplockticket.h

An advanced usage of atomic operations is found in:

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/threadqimpl.h

Given a focus on using model-checking to drive test generation, the formalism/tool most likely to deliver useful results will be Promela/SPIN. Each C11 Atomic function can be modelled as atomic in Promela, while code that uses them is modelled as non-atomic. The issue that might arise is how to get deterministic tests.

There has been work on doing formal modelling of C11 concurrency that should be explored. A brief overview of the literature follows:

Mark Batty's PhD [Bat15] used the specification language Lem to formalise atomics, with HOL4 for proofs. He also co-developed the `cppmem` tool and worked closely with the C11 committee. He also had a POPL paper [BDW16] in which the axioms of the C11 concurrency model are simplified, and which is also extended to cover OpenCL.

Viktor Vafeiadis describes a "relaxed" separation logic [VN13] to reason about programs running in the relaxed memory model. Ori Lahav and Viktor Vafeiadis [LV16] explain weak memory behaviour in terms of re-orderings and program transformations.

Norris and Demksy [ND13] developed a tool called CDSChecker that exhaustively explores concurrent code under the C/C++ concurrency model.

Robin Morisset's PhD [Mor17] looks at compiler optimisations that can be used with C/C++ concurrency. He used random testing to find bugs in GCC's implementation of atomics.

Finally there is a report by Zixian Cai, a research assistant at ANU in Australia, that uses TLAplus to verify concurrent data structures. It can be downloaded from:

https://openresearch-repository.anu.edu.au/bitstream/1885/151830/1/20181109-SCNC2102-Zixian.pdf

4.1.4 Key data-structures

Two key data-structures used in RTEMS are chains, and red-black trees. Chains are used to manages sets of things, while red-black trees are a way of keeping binary search trees balanced.

A possible first exercise might be to consider a usage of these datastructures in a context involving C11 atomics.

Code for chains can be found in:

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/chain.h

https://git.rtems.org/rtems/tree/cpukit/score/src/chain.c

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/chainimpl.h

while red-black trees are implemented in:

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/rbtree.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/rbtreeimpl.h

There are also implementations of the form `rbtreeXXX.c` in https://git.rtems.org/rtems/tree/cpukit/score/src/ for the above where `XXX` ranges over { `extract`, `insert`, `iterate`, `next`, `postorder`, `replace` }.

The red-black tree implementation also depends on `<sys.tree.h>` that defines macros for types it uses. It is not part of the RTEMS sources and is obtained from:

https://sourceware.org/git/gitweb.cgi?p=newlib-cygwin.git;a=blob;f=newlib/libc/include/sys/tree.h;h=fd66ceba1cefe37699e37b49b2c84d654031fd3b;hb=HEAD

4.1.5 RTEMS API Specifications

It should be noted that only the Classic API is in the agreed space profile [eb19b]. The official definition of the Classic API can be found at:

https://docs.rtems.org/branches/master/c-user/index.html

It would be useful to see where the code sources listed in Appendix 7 are to be found in the above API documentation.

CHAPTER

FIVE

INTEGRATION

There are two aspects to integration: the technical issue of how the formal tools join the toolchain and how their results enter the datapack; and the socio-technical issue of how the RTEMS community takes to the changes that might result to RTEMS coding practices.

5.1 Integrating with Toolchain/DataPack

The key deliverable from Tasks 1&2 for Task 3 that has been identified is the one that defines the qualification target [eb19a]. In particular a section in the "Work Items" chapter regarding requirements for build configurations, that states:

> "Build configuration requirements define what needs to be built (libraries, object files, test executables, etc.) and how (configuration option header files, compiler flags, linker flags, etc.). The goal is to generate build files (Makefile or waf) and content for the Software Configuration File (SCF) from it. A YAML scheme needs to be defined for this purpose.""

This probably needs to spelled out in more detail, but in any case we plan to to comply with these requirements. There is no obvious reason why we could not do so.

It has been suggested that the best way to achieve integration between Task 3 deliverables and those of Tasks 1 and 2 is to work iteratively with the agile process to be followed after the PDR, which will be fleshing out how the qualification toolchain will be build, and how the resulting data will be formatted for the datapack.

Another relevant part of the qualification target [eb19a] is Section 6.5 "Work Packages". This starts with a diagram showing these mapped to sprints, from Sprint 0 to Sprint 20, summarised below.

```
Sprint 0 : Build System; Software User Manual (SUM)
Sprint 1 : Application Configuration ; SUITP ; SVS for TS
Sprint 2 Object Support ; Partition Manager
Sprint 3 : Barrier Manager
Sprint 4 : Event Manager
Sprint 5 : Timer Manager
Sprint 6 : Message Queue Manager
Sprint 7 : Extension Manager
Sprint 8 : Semaphore Manager
Sprint 9 : Task Manager
Sprint 10 : Scheduler Manager
```

(continues on next page)

(continued from previous page)

```
Sprint 11 : Clock Manager
Sprint 12 : Rate Monotonic Manager
Sprint 13 : C Standard Support
Sprint 14 : System Initialization
Sprint 15 : System Termination
Sprint 16 : Software Configuration File (SCF) ; Software Reuse File (SRF)
Sprint 17 : Board Support Package ; SPARC Support
Sprint 18 : UART Driver ; GPIO Driver
Sprint 19 : SpaceWire Driver
Sprint 20 : MIL-STD-1553 Driver
```

Initially these deal with basic infrastructure such as the Build System and Application Configuration (Sprints 0-1). We can see Task 3.2 keeping in close contact at this stage, ensuring that we can fit into whatever is being developed and deployed. Of interest here is what Task 3.2 should contribute to the SUM, SUITP and SVS.

From Sprint 2 onwards, we find that Task 3.2 will diverge simply because we only consider a subset of algorithms. Additionally, the optimal order for us to address algorithms may not match that for Tasks 1.2 and 2.2 because our criteria differ. We would prefer to start with algorithms that are easy from a formal verification perspective, in order to get the tooling working properly within the toolchain, and ensure that reports so generated are of the correct form for the datapack.

We propose that once the algorithms and tools for the first phase, up to the DDR are agreed, that we see how our activities can mesh with the sprints.

5.2 RTEMS Community Integration

A series of queries were put to the RTEMS community during Task 3.1. The relevant email threads are archived and can be accessed as follows:

Code annotations:

https://lists.rtems.org/pipermail/devel/2019-September/027656.html

Algorithms to be verified:

https://lists.rtems.org/pipermail/devel/2019-August/027536.html

Installing Doorstop on OS X:

https://lists.rtems.org/pipermail/users/2019-October/067199.html

Use of docker:

https://lists.rtems.org/pipermail/users/2019-October/067236.html

Since these discussions, subsequent experience with Doorstop has led to its use being discontinued, in favour of a custom solution developed within the consortium. We discuss code annotations and Docker below, while the issue of which algorithms to verify is handled in Section 4.

5.2.1 Code Annotation

Some tools will require C-code with formal annotations on specific lines. Typical annotations include pre- and post-conditions, and loop invariants. They are usually implemented as specialised C comments. An exception to this is that the *modex* tool used to convert C code to Promela will convert C `assert()` statements into corresponding Promela assertions.

5.2.1.1 Frama-C Annotations

The format of a Frama-C ACSL annotation is

```
/*@ ...... */
```

Frama-C can obtain pre/post-conditions from either the .h or .c files. That is because it always runs a preprocessor first before doing any analysis. Loop invariants need to be placed in-situ, but we also note that quite a lot of RTEMS code is in .h files, as well as .c ones.

5.2.1.2 Annotation Schemes

In the proposal we identified three different ways to handle annotations:

FULL

> Full code annotation simply means that annotations required by tools are inserted directly into the code at appropriate points.

TAG

> The locations in code that need annotations are marked with a small discrete tag (specialised comment, or specialised part of existing comment) that uniquely identifies that code location. These tags are keys into some form of repository that associates annotations (one from each tool) with that location. Using a tool means injecting the relevant annotations into the C-code that is then passed to the tool for analysis.

GHOST

> No annotation is added to the C sources. Instead, a separate copy of the code with annotations is kept for use by tools. For this to be practical, there needs to be an automated way of mapping changes in the main C sources back into the ghost copy.

It became very clear that the GHOST proposal is unworkable. It seemed that TAG might be a good compromise regarding the level of intrusion posed by some annotations.

5.2.1.3 Community Feedback

An email, reproduced in Appendix 7.4 was sent to the devel@rtems.org mailing list. It assumed that the FULL scheme would meet resistance, and so proposed the TAG scheme, using Frama-C as an example.

The first community response focused on Frama-C and reported on issues installing it, citing in particular the large number of packages that would be required. It also asked about the provenance of the Frama-C tool.

The second community response was a high-level overview of issues in adding in new qualification tools in general. Many of these should ideally be addressed at the consortium level. However, a strong opinion was expressed that the FULL scheme is preferable to the TAG scheme, as the latter has more moving parts to go wrong, and the annotation contents require looking up a different file. However it was acknowledged that the FULL scheme could be problematic if it was decided, in the future perhaps, to use a number of tools that required code annotations, but which had different syntaxes. In this case, having a common TAG for a code location, which could then be used to access the appropriate annotation syntax.

In a telephone call with Florent Kirchner and Loic Sorrenson of CEA, who maintain Frama-C, they talked about using the Frama-C plugin facility to support the linking of annotations from a specification written in ACSL into the relevant parts of the source code for analysis. This raises the interesting question of whether it is possible to use the traceability features of the qualification toolchain to support annotation management.

Our current position, having considered and replied to the above, is to point that these annotations are most likely to only be applied to stable, critical code, and should be applied in the first instance using the FULL scheme, in a limited setting, before deciding if in fact TAG is better.

The current state of this email thread can be viewed at

https://lists.rtems.org/pipermail/devel/2019-September/027656.html

5.2.2 Docker

5.2.2.1 Using Docker

It has been agreed that the consortium will use Docker containers (https://docker.com/) to distribute all tools. The issue then naturally arises is if Docker is acceptable to the RTEMS user community. The simple answer is, it is not.

An email regarding this matter to `users@rtems.org` elicited a response that containerisation is fine for third-party material, but is not OK for mainstream RTEMS software and tools. A key principles are those of “host independence” and “reproducible builds”. Some discussion also took place on the RTEMS-SMP Technical mailing list.

The current state of

The email thread is archived at

https://lists.rtems.org/pipermail/users/2019-October/067236.html

and

https://lists.rtems.org/pipermail/users/2019-November/067269.html

The latter link is my round-up of the discussion for RTEMS users.

5.2.3 Future CI Issues

One way to handle the issue of divergence between code and annotations if the TAG scheme is used, would be simply that the standard CI process of RTEMS, expanded to re-run formal verification checks, would flag any divergences. However, this presents a long term issue for RTEMS community. This issue was flagged during the consultation with the community regarding code annotations. For example, using Frama-C requires the OCaml compiler and the `opam` package manager, which will then install over a hundred packages. The following two long-term issues arise.

1. Keeping annotations and code aligned requires a maintainer with formal expertise. Who will play this role once the RTEMS-SMP-Q activity is complete?
2. Some of the tools we might want to use can be quite costly in terms of storage and computation power. The resources available for CI at `rtems.org` are limited. Is there a long-term solution to ensure that sufficient CI computing power can be made available?

Does it make sense for there to be a separate CI platform that does the heavier qualification re-runs?

CHAPTER SIX

PLANNING

We start with a high-level overview of what we propose for Task 3.2. We then propose some specific details regarding which algorithms and tools might be deployed.

6.1 Task 3.2 Overview

For each algorithm, or collection of related algorithms, that we propose to verify, we then need to deploy those tools to work with those algorithms. This will require:

1. Integrating the tools into the toolchain.
2. Developing external models of requirements and specifications as needed.
3. Modifying the RTEMS code if required, either to avoid false positives, or because the tool requires code annotations.

Task 3.2 in effect has 3 phases of roughly equal length, punctuated by the DDR and CDR in that order. We see the phase leading up to the DDR focusing on evaluating proposed algorithm/tool combinations and getting the tools into the toolchain, and the reports into the form required for the datapack. The focus from DDR onwards would be mainly on using the established toolchain to fill out the datapack. Consideration may be given at the CDR to adjusting the algorithm/tool choices.

The precise nature of the DDR deliverables will depend on decisions regarding code scope, formalisms, approaches and tooling. Given that the DDR is notionally 5 months into a 15month Task 3.2 timeline, we should expect to have all tooling in place with the various verification processes working, and perhaps 20-25% of the code scope covered.

The CDR deliverables are due in another 5 months along. We would expect this time to be spent increasing the coverage of the targeted code, given the tools developed for the DDR.

Post CDR, leading up to QR, we might anticipate broadening the tool-base, based on how the tools chosen earlier are performing.

6.2 DDR Deliverables

The focus here is really on identifying the tools for the chosen algorithms, and getting them deployed in the toolchain with some proof of concept demonstrations to populate the datapack.

We propose that an early focus should be on the use of Promela/SPIN, because it can most easily address the modelling of atomics and synchronisation. It can also be used in a more general setting to assist with test generation.

Frama-C is worth applying to code that is more sequential in nature.

For each algorithm group already identified, we suggest which tools might be effective:

Scheduling Algorithms, Synchronisation Primitives

> Promela/SPIN, or TLAplus. SPIN is preferred because atomicity is a built-in concept in its model. Use of Frama-C is currently problematic. However, there may be parts of the scheduler that are effectively "single-threaded" in that they don't run concurrently with the tasks they are scheduling. However the SMP aspect of the scheduler along with task migration means this might not be quite so clear cut.

ContextSwitching

> Isabelle/HOL using the SPARCv8 model developed at Singapore.

Key Datastructures

> Frama-C/WP for code that does not try to be atomic. Any API calls that involve atomicity and/or synchronisation would require Promela/SPIN modelling.

RTEMS API Specifications

> Again, it depends on the expected atomicity.

6.3 CDR Deliverables

We would expect to apply the tools to generate the datapack based on progress made by the DDR and decisions made then on which algorithm/tool combinations are most worth pursuing.

The CDR review may then revise some of these decisions for the final phase of Task 3.2 up to the QR.

6.4 The Plan

Given the discussion spurred by asking the RTEMS community about Docker, the plan for Task 3.2 should be informed by the following summary of the most recent email:

Approach 1: use formal models (of requirements and/or specifications) in a behind the scenes way to produce tests. These tests would be deployed to RTEMS just like any other tests from other sources, and would be amenable to code review. The models and any tools use to produce these tests would not be part of RTEMS proper, but we would make them freely available for those who are interested.

Approach 2: use a formal tool to analyse RTEMS code, ideally as part of a standard CI/regression testing process. This would require us to ensure that such tools had a "reasonably sized" footprint, and that there was a mechanism for long-term maintenance of any code annotations.

We propose to start with approach 1 - see how far this can go. Approach 2 requires us to do some thinking and work.

6.5 Timeline

Kick-off: Feb 13th 2019

Table 1: Reviews and Milestones

What	Date	M/S	Deliverable	R/B?
KO	2019-02-13			
SRR/SwRR	2019-07-10			
PDR	2019-10-09	MS1	Algorithm Selection	B
DDR#1	2020-07-09		Formal Verification Execution	R
DDR#2	2020-10-16	MS2	Formal Verification Execution	R
CDR	2021-01-21	MS3	Formal Verification Execution	R
QR	2021-04-01		Formal Verification Execution	R
FR/AR	2021-05-15	MS4	Formal Verification Execution	B
FR/AR	2021-05-15	MS4	Verification Report	B

R : Review, B : Baseline

CHAPTER

SEVEN

APPENDIX

7.1 Conference Call with CEA

A conference call was held with Frama-C project members, working for CEA-LIST (http://www-list.cea.fr), on Wed 18th September 2019, 3pm IST, regarding prospects for its use with RTEMS-SMP. Here are some notes covering the key points.

7.1.1 Attendance

Andrew Butterfield, Lero@TCD, Leading Formal Verification task of RTEMS-SMP

Florent Kirchner, CEA-LIST Head of Lab developing Frama-C. (software verification & analysis, abstract interpretation, cyber-security).

Loic Correnson, CEA-LIST, Proof team (focus on the WP plugin).

7.1.2 Summary

The Frama-C team has about 10 people on projects, most of them funded by contracts.

After some introductions and sharing of biographies, I gave a short overview of the RTEMS-SMP project, and how we hoped to deploy formal techniques. The Frama-C team expressed a lot of interest in the project, and agreed that its unusual mix of traditional safety critical software development with both agile techniques and open-source community would produce some interesting challenges. They recommended that we keep in touch with them as we explored the use of Frama-C, as they could advise us regarding practical uses when trying to use their tools.

A number of discussions ensured regarding various aspects of how Frama-C might be used. The focus was on the WP plugin of Frama-C, which is the plugin that uses the ACSL annotations, and which connects to a range of theorem provers. In no particular order:

1. They said that they had applied WP to a range of code types, including the kernel of an operating system called ConTiki.
2. They have ongoing work looking at how best to integrate Frama-C into CI systems.
3. Airbus re-run the WP tool to check code proofs on a nightly basis over their code.

4. Regarding keeping ACSL annotations separate from the code (like TAG), they said that this could be managed by a Frama-C plugin (which would have to be written in OCaml).
5. When asked why their "pathcrawler" plugin to do automatic test generation was not easily available, they explained that it was because DO178C forbids the use of a high-level design artifact to generate tests for a low-level one. They require that all tests be developed independently. Given that restriction they decided not to put too much effort into it. An online demo version is available (http://pathcrawler-online.com:8080).
6. There was an M.Sc. thesis that applied Frama-C/WP to RTEMS back in 2009 [daSCarloto10], but they said that there had been many improvements to the tool since.
7. Regarding C11 support in general, and C11 Atomics in particular, they didn't see any issue

While they are keen to work with us in some capacity as this activity moves on, they did point out that they have limited resources to devote to addressing issues we raise.

7.1.3 Follow-Up

An email has been sent on October 14th to the Frama-C people to enquire about WP usage at Airbus, as suggested by RID PDR-TT-02.

We got a reply indicating that a paper on this topic was published at ERTS2 2018 [BDE+18], which can be found at

https://www.erts2018.org/authors_detail_inverted_Brahmi%20Abderrahmane.html

and which contains contact details for all the authors, including three from Airbus. These are

Abderrahmane Brahmi, David Delmas, Mohamed Habib Essoussi

accessible using email addresses of the form First.Last@airbus.com

7.2 Context-Switch Assembler

The following assembly files have been suggested for consideration:

https://git.rtems.org/rtems/tree/bsps/sparc/shared/start/start.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/cpu_asm.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/syscall.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/window.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/sparc-access.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/sparc-context-volatile-clobber.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/sparc-counter-asm.S

https://git.rtems.org/rtems/tree/cpukit/score/cpu/sparc/sparc-context-validate.S

https://git.rtems.org/rtems/tree/bsps/sparc/leon3/start/bspidle.S

7.3 Sources Using Atomics

The source files that use Atomics are:

https://git.rtems.org/rtems/tree/cpukit/libtrace/record/record.c

https://git.rtems.org/rtems/tree/cpukit/posix/src/mutexunlock.c

https://git.rtems.org/rtems/tree/cpukit/posix/src/mutexunlock.c

https://git.rtems.org/rtems/tree/cpukit/libcsupport/src/open.c

https://git.rtems.org/rtems/tree/cpukit/libcsupport/src/close.c

https://git.rtems.org/rtems/tree/cpukit/include/rtems/record.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/libio.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/test.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smpimpl.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/threaddispatch.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smpbarrier.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/threadqimpl.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smplockmcs.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smplockticket.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/threadimpl.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/score/smplockseq.h

https://git.rtems.org/rtems/tree/cpukit/include/rtems/libio_.h.

https://git.rtems.org/rtems/tree/cpukit/score/src/libatomic.c

https://git.rtems.org/rtems/tree/cpukit/score/src/once.c

https://git.rtems.org/rtems/tree/cpukit/score/src/smpmulticastaction.c

https://git.rtems.org/rtems/tree/cpukit/score/src/smpbarrierwait.c

https://git.rtems.org/rtems/tree/cpukit/score/src/smp.c

https://git.rtems.org/rtems/tree/cpukit/score/src/kern_tc.c

https://git.rtems.org/rtems/tree/cpukit/libtest/testparallel.c

7.4 Code Annotations

```
Dear RTEMS developers,

as some of you may be aware, I am leading a task as part of the RTEMS
Qualification activity that explores the use of formal methods to assist with
code verifications.

Some of these techniques work best if annotations are added to source code, but
these annotations are *not* lightweight.

I am fairly sure that having these annotations in code is not going to be
viable, and neither is maintaining two code versions, one with annotations, and
one without.

PROPOSAL (part 1):

Lightweight annotation - that just identifies the relevant code point, with a
separate file that provides the full annotation. We add an annotation
pre-processor step that merges the two before handing off to the relevant tool.
(part 2 shows how this might work in an example below: `max_seq`)

We would all be very interested in your feedback.

The rest of this email shows some concrete examples to give you some idea of
what is involved (based on a tool called Frama-C that has been used by Airbus,
among others).

A header file with pre-/post condition annotations:
 swap.h:
       /*@ requires \valid(a) && \valid(b);
       @ ensures A: *a == \old(*b) ;
       @ ensures B: *b == \old(*a) ;
       @ assigns *a,*b ;
       @*/
     void swap(int *a,int *b) ;

The implementation, *in this case*,  needs no further annotations

 swap.c:
     #include "swap.h"

     void swap(int *a,int *b)
     {
       int tmp = *a ;
       *a = *b ;
       *b = tmp ;
       return ;
     }

It illustrates that annotations associated with a C function (pre- and
post-conditions) can be put in the header file, which is where you currently use
Doxygen to document the function call. This is probably fairly low impact from
your perspective.
```

(continues on next page)

(continued from previous page)

```
However, if the implementation code contains loops, then we need annotations in
the code at those loops. The following, from the Frama-C ACSL Tutorial
(https://frama-c.com/acsl_tutorial_index.html) shows the annotations required to
verify the correctness of a function that finds the maximum of a sequence of
integers.

 maxseq.c:
       int max_seq(int* p, int n) {
         int res = *p;
         //@ ghost int e = 0;
         /*@ loop invariant \forall integer j;
                  0 <= j < i ==> res >= \at(p[j],Pre);
             loop invariant
                  \valid(\at(p,Pre)+e) && \at(p,Pre)[e] == res;
             loop invariant 0<=i<=n;
             loop invariant p==\at(p,Pre)+i;
             loop invariant 0<=e<n;
         */
         for(int i = 0; i < n; i++) {
           if (res < *p) {
             res = *p;
             //@ ghost e = i;
           }
           p++;
         }
         return res;
       }

20 lines of code, of which 10 are annotations.

PROPOSAL (part 2):

We have maxseq.a:
   maxseq.loop
   //@ ghost int e = 0;
   /*@ loop invariant \forall integer j;
           0 <= j < i ==> res >= \at(p[j],Pre);
      loop invariant
           \valid(\at(p,Pre)+e) && \at(p,Pre)[e] == res;
      loop invariant 0<=i<=n;
      loop invariant p==\at(p,Pre)+i;
      loop invariant 0<=e<n;
   */
   maxseq.body
     //@ ghost e = i;

and maxseq.c looks like:
       int max_seq(int* p, int n) {
         int res = *p;
         //@ maxseq.loop
         for(int i = 0; i < n; i++) {
           if (res < *p) {
```

(continues on next page)

(continued from previous page)

```
            res = *p;
            //@ maxseq.body
          }
          p++;
        }
        return res;
      }

Again, feedback would be much appreciated!
```

BIBLIOGRAPHY

[AP18] Gul Agha and Karl Palmskog. A survey of statistical model checking. *ACM Trans. Model. Comput. Simul.*, 28(1):6:1–6:39, 2018. URL: https://doi.org/10.1145/3158668, doi:10.1145/3158668.

[BDW16] Mark Batty, Alastair F. Donaldson, and John Wickerson. Overhauling SC atomics in C11 and opencl. In Rastislav Bod'ık and Rupak Majumdar, editors, *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, 634–648. ACM, 2016. URL: https://doi.org/10.1145/2837614.2837637, doi:10.1145/2837614.2837637.

[Bat15] Mark John Batty. *The C11 and C++11 concurrency model*. PhD thesis, University of Cambridge, UK, 2015. URL: http://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.708458.

[BDE+18] Abderrahmane Brahmi, David Delmas, Mohamed Habib Essoussi, Famantanantsoa Randimbivololona, Abdellatif Atki, and Thomas Marie. Formalise to automate: deployment of a safe and cost-efficient process for avionics software. In *9th European Congress on Embedded Real Time Software and Systems (ERTS 2018)*. Toulouse, France, January 2018. URL: https://hal.archives-ouvertes.fr/hal-01708332.

[Bra13] Björn B. Brandenburg. A Fully Preemptive Multiprocessor Semaphore Protocol for Latency-Sensitive Real-Time Applications. In *Proceedings of the 25th Euromicro Conference on Real-Time Systems (ECRTS 2013)*, 292–302. 2013. URL: http://www.mpi-sws.org/~bbb/papers/pdf/ecrts13b.pdf.

[BW13] A. Burns and A. J. Wellings. A Schedulability Compatible Multiprocessor Resource Sharing Protocol - MrsP. In *Proceedings of the 25th Euromicro Conference on Real-Time Systems (ECRTS 2013)*. 2013. URL: http://www-users.cs.york.ac.uk/~burns/MRSPpaper.pdf.

[dVT00] René G. de Vries and Jan Tretmans. On-the-fly conformance testing using spin. *International Journal on Software Tools for Technology Transfer*, 2(4):382–393, Mar 2000. URL: https://doi.org/10.1007/s100090050044, doi:10.1007/s100090050044.

[ECS09] ECSS. *ECSS-E-ST-40C - Software general requirements*. European Cooperation for Space Standardization, 2009. URL: https://ecss.nl/standard/ecss-e-st-40c-software-general-requirements/.

[ECS17] ECSS. *ECSS-Q-ST-80C Rev.1 - Software product assurance*. European Co-operation for Space Standardization, 2017. URL: https://ecss.nl/standard/ecss-q-st-80c-rev-1-software-product-assurance-15-february-2017/.

[eb19a] embedded brains. *Technical Note: RTEMS SMP Qualification Target, Release 1*. embedded brains GmbH, 2019.

[eb19b] embedded brains. *Technical Note: Space Profile, Release 2*. embedded brains GmbH, 2019.

[FKDO12] Ling Fang, Takashi Kitamura, Thi Bich Ngoc Do, and Hitoshi Ohsaki. Formal model-based test for AUTOSAR multicore RTOS. In Giuliano Antoniol, Antonia Bertolino, and Yvan Labiche, editors, *Fifth IEEE International Conference on Software Testing, Verification and Validation, ICST 2012, Montreal, QC, Canada, April 17-21, 2012*, 251–259. IEEE Computer Society, 2012. URL: https://doi.org/10.1109/ICST.2012.105, doi:10.1109/ICST.2012.105.

[FGT11] Antonio Filieri, Carlo Ghezzi, and Giordano Tamburrelli. Run-time efficient probabilistic model checking. In Richard N. Taylor, Harald C. Gall, and Nenad Medvidovic, editors, *Proceedings of the 33rd International Conference on Software Engineering, ICSE 2011, Waikiki, Honolulu , HI, USA, May 21-28, 2011*, 341–350. ACM, 2011. URL: https://doi.org/10.1145/1985793.1985840, doi:10.1145/1985793.1985840.

[Fox15] Anthony C. J. Fox. Improved tool support for machine-code decompilation in HOL4. In Christian Urban and Xingyuan Zhang, editors, *Interactive Theorem Proving - 6th International Conference, ITP 2015, Nanjing, China, August 24-27, 2015, Proceedings*, volume 9236 of Lecture Notes in Computer Science, 187–202. Springer, 2015. URL: https://doi.org/10.1007/978-3-319-22102-1\T1\textbackslash{}_12, doi:10.1007/978-3-319-22102-1_12.

[Gau05] Marie-Claude Gaudel. Formal methods and testing: hypotheses, and correctness approximations. In *Proceedings of the 2005 International Conference on Formal Methods*, FM'05, 2–8. Berlin, Heidelberg, 2005. Springer-Verlag. URL: http://dx.doi.org/10.1007/11526841_2, doi:10.1007/11526841_2.

[Gau95] Marie-Claude Gaudel. Testing can be formal, too. In Peter D. Mosses, Mogens Nielsen, and Michael I. Schwartzbach, editors, *TAPSOFT'95: Theory and Practice of Software Development, 6th International Joint Conference CAAP/FASE, Aarhus, Denmark, May 22-26, 1995, Proceedings*, volume 915 of Lecture Notes in Computer Science, 82–96. Springer, 1995. URL: https://doi.org/10.1007/3-540-59293-8\T1\textbackslash{}_188, doi:10.1007/3-540-59293-8_188.

[HBB+09] Robert M. Hierons, Kirill Bogdanov, Jonathan P. Bowen, Rance Cleaveland, John Derrick, Jeremy Dick, Marian Gheorghe, Mark Harman, Kalpesh Kapoor, Paul J. Krause, Gerald Lüttgen, Anthony J. H. Simons, Sergiy A. Vilkomir, Martin R. Woodward, and Hussein Zedan. Using formal specifications to support testing. *ACM Comput. Surv.*, 41(2):9:1–9:76, 2009. URL: https://doi.org/10.1145/1459352.1459354, doi:10.1145/1459352.1459354.

[HSananTL16] Zhe Hou, David Sanán, Alwen Tiu, and Yang Liu. A formal model for the sparcv8 ISA and a proof of non-interference for the LEON3 processor. *Archive of Formal Proofs*, 2016. URL: https://www.isa-afp.org/entries/SPARCv8.shtml.

[HSananT+16] Zhe Hou, David Sanán, Alwen Tiu, Yang Liu, and Koh Chuen Hoa. An executable formalisation of the sparcv8 instruction set architecture: A case study for the LEON3 processor. In John S. Fitzgerald, Constance L. Heitmeyer, Stefania Gnesi, and Anna Philippou, editors, *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, volume 9995 of Lecture Notes in Computer Science, 388–405. 2016. URL: https://doi.org/10.1007/978-3-319-48989-6\T1\textbackslash{}_24, doi:10.1007/978-3-319-48989-6_24.

[HeraultLMP04] Thomas Hérault, Richard Lassaigne, Frédéric Magniette, and Sylvain Peyronnet. Approximate probabilistic model checking. In Bernhard Steffen and Giorgio Levi, editors, *Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings*, volume 2937 of Lecture Notes in Computer Science, 73–84. Springer, 2004. URL: https://doi.org/10.1007/978-3-540-24622-0\T1\textbackslash{}_8, doi:10.1007/978-3-540-24622-0_8.

[KEH+09] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. Sel4: formal verification of an OS kernel. In Jeanna Neefe Matthews and Thomas E. Anderson, editors, *Proceedings of the 22nd ACM Symposium on Operating Systems Principles 2009, SOSP 2009, Big Sky, Montana, USA, October 11-14, 2009*, 207–220. ACM, 2009. URL: https://doi.org/10.1145/1629575.1629596, doi:10.1145/1629575.1629596.

[KNP02] Marta Z. Kwiatkowska, Gethin Norman, and David Parker. PRISM: probabilistic symbolic model checker. In Tony Field, Peter G. Harrison, Jeremy T. Bradley, and Uli Harder, editors, *Computer Performance Evaluation, Modelling Techniques and Tools 12th International Conference, TOOLS 2002, London, UK, April 14-17, 2002, Proceedings*, volume 2324 of Lecture Notes in Computer Science, 200–204. Springer, 2002. URL: https://doi.org/10.1007/3-540-46029-2\T1\textbackslash{}_13, doi:10.1007/3-540-46029-2_13.

[LV16] Ori Lahav and Viktor Vafeiadis. Explaining relaxed memory models with program transformations. In John S. Fitzgerald, Constance L. Heitmeyer, Stefania Gnesi, and Anna Philippou, editors, *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, volume 9995 of Lecture Notes in Computer Science, 479–495. 2016. URL: https://doi.org/10.1007/978-3-319-48989-6\T1\textbackslash{}_29, doi:10.1007/978-3-319-48989-6_29.

[Mor17] Robin Morisset. *Compiler optimisations and relaxed memory consistency models. (Optimisations des compilateurs et modèles mémoire relâchés)*. PhD thesis, PSL Research University, Paris, France, 2017. URL: https://tel.archives-ouvertes.fr/tel-01823521.

[MGS12] Magnus O. Myreen, Michael J. C. Gordon, and Konrad Slind. Decompilation into logic - improved. In Gianpiero Cabodi and Satnam Singh, editors, *Formal Methods in Computer-Aided Design, FMCAD 2012, Cambridge, UK, October 22-25, 2012*, 78–81. IEEE, 2012.

[ND13] Brian Norris and Brian Demsky. Cdschecker: checking concurrent data structures written with C/C++ atomics. In Antony L. Hosking, Patrick Th. Eugster, and Cristina V. Lopes, editors, *Proceedings of the 2013 ACM SIGPLAN Interna-*

tional Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013, 131–150. ACM, 2013. URL: https://doi.org/10.1145/2509136.2509514, doi:10.1145/2509136.2509514.

[SP16] Subash Shankar and Gilbert Pajela. A tool integrating model checking into a C verification toolset. In Dragan Bosnacki and Anton Wijs, editors, *Model Checking Software - 23rd International Symposium, SPIN 2016, Co-located with ETAPS 2016, Eindhoven, The Netherlands, April 7-8, 2016, Proceedings*, volume 9641 of Lecture Notes in Computer Science, 214–224. Springer, 2016. URL: https://doi.org/10.1007/978-3-319-32582-8\T1\textbackslash{}_15, doi:10.1007/978-3-319-32582-8_15.

[UAHB10] Andreas Ulrich, El Hachemi Alikacem, Hesham Hallal, and Sergiy Boroday. From scenarios to test implementations via promela. In Alexandre Petrenko, Adenilso da Silva Simão, and José Carlos Maldonado, editors, *Testing Software and Systems - 22nd IFIP WG 6.1 International Conference, ICTSS 2010, Natal, Brazil, November 8-10, 2010. Proceedings*, volume 6435 of Lecture Notes in Computer Science, 236–249. Springer, 2010. URL: https://doi.org/10.1007/978-3-642-16573-3_17, doi:10.1007/978-3-642-16573-3_17.

[VN13] Viktor Vafeiadis and Chinmay Narayan. Relaxed separation logic: a program logic for C11 concurrency. In Antony L. Hosking, Patrick Th. Eugster, and Cristina V. Lopes, editors, *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013*, 867–884. ACM, 2013. URL: https://doi.org/10.1145/2509136.2509532, doi:10.1145/2509136.2509532.

[daSCarloto10] Carlos José Abreu Dias da Silva Carloto. Towards a formally verified microkernel using the frama-c tools. Master's thesis, Computer Science Department, University of Beira Interior, Covilhã, Portuga, 2010. URL: https://pdfs.semanticscholar.org/da53/874d44a72a80cb63f308af165b682a063f18.pdf.